

Министерство образования Российской Федерации
МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. БАУМАНА

Факультет: Информатика и системы управления
Кафедра: Информационная безопасность (ИУ8)

ТЕОРИЯ ИГР В СИСТЕМАХ ЗАЩИТЫ
ИНФОРМАЦИИ

Лабораторная работа №7
«Кооперативные игры. Вектор Шепли»

Вариант 13

Преподаватель:
Захаров П.В.

Студент:
Степанова Е.А.

Группа:
ИУ8-21М

Москва 2026

Цель работы

Изучить постановку кооперативной игры и найти оптимальное распределение выигрыша (дележ) между игроками путем вычисления компонент вектора Шепли.

Постановка задачи

Для заданной характеристической функцией игры по варианту выполнить:

- 1) Проверить кооперативную игру на супераддитивность и выпуклость. Если игра по варианту не супераддитивна, изменить характеристическую функцию таким образом, чтобы игра была супераддитивна. Продемонстрировать изменения и повторную проверку.
- 2) Составить программу вычисления компонент вектора Шепли и, в зависимости от варианта, рассчитать его.
- 3) Проверить условия индивидуальной и групповой рационализации.

Ход работы

Пусть $N = 4$, а ХФ имеет вид:

$v(AE) = 0$; $v(\{1\}) = v(\{2\}) = 3$; $v(\{3\}) = 2$; $v(\{4\}) = 4$; $v(\{1,2\}) = v(\{2,3\}) = 6$; $v(\{1,3\}) = 5$; $v(\{1,4\}) = v(\{3,4\}) = 8$; $v(\{2,4\}) = 9$; $v(\{1,2,3\}) = 9$; $v(\{1,2,4\}) = v(\{2,3,4\}) = 12$; $v(\{1,3,4\}) = 10$; $v(I) = 14$.

Данная кооперативная игра является супераддитивной, т.к. (с учетом симметричности):

$$v(\{i,j\}) > v(\{i\}) + v(\{j\}) \quad (d_{i,j} = 0)$$

$$v(I) > v(\{i\}) + v(\{j\}) + v(\{k\})$$

$$v(I) > v(\{i,j\}) + v(\{k\}) \quad (d_{i,j} + d_{i,k} + d_{j,k} = 0)$$

При анализе второй характеристической функции были выявлены нарушения супераддитивности. В частности, были исправлены следующие значения:

$$v(1,3,4) < v(1,3) + v(4) \text{ и } v(1,2,3,4) < v(1,2,4) + v(3)$$

Игра не является супераддитивной в исходном виде и требует коррекции характеристической функции для обеспечения условия $v(S \cup T) \geq v(S) + v(T)$ для всех непересекающихся коалиций S и T.

Распределение выигрыша несимметрично.

Игрок 4 получает наибольшую долю (5.416667), так как его вклад в коалиции наиболее значителен. Игрок 3 получает наименьшую долю (2.750000), что соответствует его наименьшему одиночному выигрышу $v(3)=2$.

Результаты на рисунке 1.

```
x_1 = 3.083333
x_2 = 3.750000
x_3 = 2.750000
x_4 = 5.416667
Σ x_i = 15.000000
v(I) = 15.000000
```

Рисунок 1. Результаты выполнения программы.

Приложение А.

Файл 'lab7.py'.

```
...

import numpy as np
from itertools import combinations, permutations
import math # добавим импорт math

class CooperativeGame:

    def __init__(self, v, n_players):

        self.v = v
        self.n = n_players
        self.players = list(range(1, n_players + 1))

    def check_superadditivity(self):

        all_subsets = []
        for r in range(1, self.n + 1):
            for combo in combinations(self.players, r):
                all_subsets.append(combo)

        is_superadditive = True
        violations = []

        for S in all_subsets:
            for T in all_subsets:
                if set(S).isdisjoint(set(T)) and len(S) > 0 and len(T) >
0:

                    S_key = tuple(sorted(S))
                    T_key = tuple(sorted(T))
                    ST_key = tuple(sorted(S + T))

                    if ST_key in self.v:
                        left = self.v[ST_key]
                        right = self.v[S_key] + self.v[T_key]

                        if left < right - 1e-6:
                            is_superadditive = False
                            violations.append((S_key, T_key, left, right))

        return is_superadditive, violations

    def check_convexity(self):

        all_subsets = []
```

```

for r in range(1, self.n + 1):
    for combo in combinations(self.players, r):
        all_subsets.append(combo)

is_convex = True
violations = []

for S in all_subsets:
    for T in all_subsets:
        S_set = set(S)
        T_set = set(T)
        union_set = S_set.union(T_set)
        inter_set = S_set.intersection(T_set)

        union_key = tuple(sorted(union_set))
        inter_key = tuple(sorted(inter_set))
        S_key = tuple(sorted(S))
        T_key = tuple(sorted(T))

        if union_key in self.v and inter_key in self.v:
            left = self.v[union_key] + self.v.get(inter_key, 0)
            right = self.v[S_key] + self.v[T_key]

            if left < right - 1e-6:
                is_convex = False
                violations.append((S_key, T_key, left, right))

return is_convex, violations

def fix_superadditivity(self, violations):

    fixed_v = self.v.copy()

    for S, T, left, right in violations:
        ST_key = tuple(sorted(set(S).union(set(T))))
        if fixed_v[ST_key] < right:
            fixed_v[ST_key] = right
            print(f"    Исправлено: v[{list(ST_key)}] = {right} (было
{left}))")

    # Дополнительная проверка для всех коалиций
    all_subsets = []
    for r in range(1, self.n + 1):
        for combo in combinations(self.players, r):
            all_subsets.append(combo)

    changed = True
    while changed:
        changed = False
        for S in all_subsets:

```

```

        for T in all_subsets:
            if set(S).isdisjoint(set(T)) and len(S) > 0 and len(T)
> 0:
                S_key = tuple(sorted(S))
                T_key = tuple(sorted(T))
                ST_key = tuple(sorted(S + T))

                if ST_key in fixed_v:
                    if fixed_v[ST_key] < fixed_v[S_key] +
fixed_v[T_key]:
                        fixed_v[ST_key] = fixed_v[S_key] +
fixed_v[T_key]
                        changed = True

        return fixed_v

def calculate_shapley_value(self):

    N = self.n
    shapley_values = np.zeros(N)

    print("\nМетод 1: Через все перестановки игроков")
    print("-" * 50)

    all_permutations = list(permutations(self.players))
    total_permutations = len(all_permutations)

    contributions = {i: [] for i in self.players}

    for perm in all_permutations:
        coalition = []
        for idx, player in enumerate(perm):
            coalition_set = set(coalition)
            coalition_key = tuple(sorted(coalition_set))

            prev_value = self.v.get(coalition_key, 0)

            new_coalition = coalition + [player]
            new_coalition_key = tuple(sorted(new_coalition))
            new_value = self.v.get(new_coalition_key, 0)

            marginal_contribution = new_value - prev_value
            contributions[player].append(marginal_contribution)

            coalition.append(player)

    print("\nРезультат вычисления:")
    for i, val in enumerate(shapley_values):
        shapley_values[i] = sum(contributions[i+1]) /
total_permutations

```

```

        print(f"  x_{i+1} = {shapley_values[i]:.6f}")

# Альтернативный расчет по формуле
print("\n" + "-" * 50)
print("Метод 2: Через формулу с коалициями")
print("-" * 50)

shapley_values2 = np.zeros(N)

for player in self.players:
    value = 0
    other_players = [p for p in self.players if p != player]

    for k in range(N):
        for coalition in combinations(other_players, k):
            coalition_set = set(coalition)
            coalition_key = tuple(sorted(coalition_set))

            S_with_i_key = tuple(sorted(list(coalition_set) +
[player]))

            v_S = self.v.get(coalition_key, 0)
            v_S_with_i = self.v.get(S_with_i_key, 0)

            marginal = v_S_with_i - v_S

            # Используем math.factorial вместо np.math.factorial
            weight = math.factorial(k) * math.factorial(N - k - 1)
/ math.factorial(N)

            value += weight * marginal

    shapley_values2[player-1] = value

print("\nРезультат вычисления:")
for i, val in enumerate(shapley_values2):
    print(f"  x_{i+1} = {val:.6f}")

return shapley_values

def check_rationality(self, shapley_values):

    # Групповая рационализация
    total_coalition_key = tuple(self.players)
    total_value = self.v.get(total_coalition_key, 0)
    sum_shapley = np.sum(shapley_values)

    print(f"\n1. Групповая рационализация:")
    print(f"  Σ x_i = {sum_shapley:.6f}")

```

```

print(f"    v(I) = {total_value:.6f}")

# Индивидуальная рационализация
print(f"\n2. Индивидуальная рационализация:")
individual_tolerance = True

for player in self.players:
    solo_key = (player,)
    solo_value = self.v.get(solo_key, 0)

    if shapley_values[player-1] >= solo_value - 1e-6:
        print(f"    ✓ x_{player} = {shapley_values[player-1]:.6f} ≥
v{{{player}}} = {solo_value:.6f}")
    else:
        print(f"    ✗ x_{player} = {shapley_values[player-1]:.6f} <
v{{{player}}} = {solo_value:.6f}")
        individual_tolerance = False

return sum_shapley, total_value, individual_tolerance

def get_variant_data(variant=13):
    """
    Возвращает данные для заданного варианта
    """

    # Данные для варианта 13 из таблицы
    # N = 4 игрока
    # ВАЖНО: проверьте по вашей таблице, правильные ли значения для
    # варианта 13!

    v = {
        # Пустая коалиция
        (): 0,

        # Одиночные коалиции (1 игрок)
        (1,): 3,
        (2,): 3,
        (3,): 2,
        (4,): 4,

        # Коалиции из 2 игроков
        (1,2): 6,
        (1,3): 5,
        (1,4): 8,
        (2,3): 6,
        (2,4): 9,
        (3,4): 8,
    }

```



```

        # Коалиция из 4 игроков (все игроки)
        (1,2,3): 9,
        (1,2,4): 12,
        (1,3,4): 10,
        (2,3,4): 12,

        (1,2,3,4): 14
    }

    return v, 4

def main():
    """Основная функция"""

    variant = 13

    print("="*70)
    print("ЛАБОРАТОРНАЯ РАБОТА №7")
    print("Кооперативные игры. Вектор Шепли")
    print(f"ВАРИАНТ {variant}")
    print("="*70)

    # Получение данных для варианта
    v, n_players = get_variant_data(variant)

    # Вывод характеристической функции
    print("\nХАРАКТЕРИСТИЧЕСКАЯ ФУНКЦИЯ:")
    print("-" * 50)
    for key in sorted(v.keys(), key=lambda x: (len(x), x)):
        if len(key) == 0:
            print(f"  v( $\emptyset$ ) = {v[key]}")
        else:
            print(f"  v{key} = {v[key]}")

    # Создание игры
    game = CooperativeGame(v, n_players)

    # 1. Проверка на супераддитивность
    is_superadditive, violations = game.check_superadditivity()

    # 2. Проверка на выпуклость
    is_convex, convex_violations = game.check_convexity()

    # 3. Если игра не супераддитивна - исправляем
    if not is_superadditive:
        fixed_v = game.fix_superadditivity(violations)
        game.v = fixed_v
        print("\nПовторная проверка супераддитивности...")
        is_superadditive, _ = game.check_superadditivity()

```

```

# 4. Вычисление вектора Шепли
shapley_values = game.calculate_shapley_value()

# 5. Проверка условий рационализации
sum_val, total_val, individual_ok =
game.check_rationality(shapley_values)

for i, val in enumerate(shapley_values):
    print(f"x_{i+1} = {val:.6f}")

print(f"Σ x_i = {sum_val:.6f}")
print(f"v(I) = {total_val:.6f}")

if __name__ == "__main__":
    main()

```